30-DAY DSA CHALLENGE • ZERO TO INTERMEDIATE

# Day 01: Foundations, Complexity & Array Basics

Comprehensive study notes covering core theory, complexity analysis, and 6 fundamental array problems in JavaScript.

**Language:** JavaScript (ES6+) **Topic:** Arrays & Complexity **Repository:** Day-01/notes.md

## 1. Core Concepts

### What is DSA?

- **Data Structures (DS):** Systematic formats for organizing, managing, and storing data to enable efficient access and modifications. (Analogy: A neatly organized toolchest).
- **Algorithms (A):** Step-by-step procedures or precise set of rules designed to solve a specific problem or compute a result. (Analogy: A recipe to bake a cake).

### Data Structure vs. Algorithm

| Feature | Data Structure | Algorithm |
|---|---|---|
| **Definition** | Data organization and storage format | Step-by-step problem-solving sequence |
| **Focus** | How data is arranged in memory | How data is manipulated or transformed |
| **Example** | Arrays, Objects, Linked Lists, Trees | Searching, Sorting, Traversing |
| **Real-World Analogy** | A library bookshelf arrangement | The method used to search for a book |

### Time Complexity Basics

Time complexity measures how the execution time of an algorithm scales as the input size ($n$) grows. Expressed using **Big O Notation**.

> **Growth Rate Hierarchy (Best to Worst)**
>
> `O(1)` < `O(log n)` < `O(n)` < `O(n²)`

- `O(1)` — **Constant Time:** Execution time stays constant regardless of input size. *Example: Accessing an array element by index (`arr[0]`).*
- `O(log n)` — **Logarithmic Time:** Execution time grows logarithmically; input size halves at each step. *Example: Binary Search.*
- `O(n)` — **Linear Time:** Execution time grows proportionally with input size. *Example: Iterating over an array once.*
- `O(n²)` — **Quadratic Time:** Execution time grows quadratically. *Example: Nested loops checking all pairs.*

### Space Complexity Basics

Space complexity measures the total auxiliary memory an algorithm requires relative to input size ($n$).

- `O(1)` **Auxiliary Space:** Memory usage stays constant regardless of input size (e.g., standard variables like `sum` or `count`).

- `O(n)` **Auxiliary Space:** Memory scales linearly with input size (e.g., creating a duplicate array of size $n$).

## 2. Introduction to Arrays

An **Array** is a linear data structure storing elements in continuous memory locations. In JavaScript, arrays are dynamic and zero-indexed.

### Indexing & Access

Accessing any element by index operates in $O(1)$ constant time.

```
const numbers = [10, 20, 30, 40];
console.log(numbers[0]); // 10 (First element)
console.log(numbers[2]); // 30 (Third element)
```

### Array Traversal

Visiting every element in an array sequentially using a loop.

```
for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}
```

## 3. The 6-Step DSA Problem-Solving Framework

1. **Understand the Problem:** Clarify constraints, inputs, outputs, and edge cases.
2. **Find the Approach:** Brainstorm brute-force first, then identify optimization patterns.
3. **Dry Run:** Trace the algorithm step-by-step using a small sample input manually.
4. **Write Code:** Write clean, structured JavaScript code.
5. **Test:** Validate against standard inputs and edge cases (empty array, single element, negative numbers).
6. **Analyze Complexity:** Evaluate and document Time and Space complexity.

## 4. Core Array Patterns Learned Today

- **Traversal Pattern:** Iterating through elements from index 0 to $n - 1$.
- **Comparison Pattern:** Tracking an extreme value (e.g., max/min) by comparing each element against a current standard.
- **Accumulator Pattern:** Accumulating values into a single running balance (e.g., total sum).
- **Counter Pattern:** Incrementing a tally whenever an element satisfies a condition.

## 5. Day 1 Coding Problems

### Problem 1: Find the Largest Element in an Array

**Problem Statement:** Given an array of numbers, return the largest element in the array.

**Approach:** Initialize `max` with `arr[0]`. Iterate from index 1 to the end. If `arr[i] > max`, update `max`.

> **Dry Run (Input: [4, 12, 7, 25, 3])**
>
> Start: `max = 4` • Index 1: `12 > 4` → `max = 12` • Index 2: `7 < 12` • Index 3: `25 > 12` → `max = 25` • Index 4: `3 < 25` → **Output: 25**

```
function findLargest(arr) {
  if (arr.length === 0) return null;
  let max = arr[0];
  for (let i = 1; i < arr.length; i++) {
    if (arr[i] > max) {
      max = arr[i];
    }
  }
  return max;
}
```

**Time Complexity:** *O(n)* **Space Complexity:** *O(1)*

**Key Learning:** Always initialize `max` with `arr[0]` instead of `0` to handle arrays with all negative values correctly.

### Problem 2: Find the Smallest Element in an Array

**Problem Statement:** Given an array of numbers, return the smallest element in the array.

**Approach:** Initialize `min` with `arr[0]`. Loop through remaining elements. If `arr[i] < min`, update `min`.

> **Dry Run (Input: [18, 5, 92, -3, 11])**
>
> Start: `min = 18` • Index 1: `5 < 18` → `min = 5` • Index 3: `-3 < 5` → `min = -3` → **Output: -3**

```
function findSmallest(arr) {
  if (arr.length === 0) return null;
  let min = arr[0];
  for (let i = 1; i < arr.length; i++) {
    if (arr[i] < min) {
      min = arr[i];
    }
  }
  return min;
}
```

**Time Complexity:** *O(n)* **Space Complexity:** *O(1)*

**Key Learning:** The dual of finding maximums; relies on updating a running baseline when encountering smaller values.

### Problem 3: Find the Sum of Array Elements

**Problem Statement:** Given an array of numbers, calculate and return the sum of all elements.

**Approach:** Initialize `sum = 0`. Loop through all elements and add each to `sum`.

```
function findSum(arr) {
  let sum = 0;
  for (let i = 0; i < arr.length; i++) {
    sum += arr[i];
  }
  return sum;
}
```

**Time Complexity:** *O(n)* **Space Complexity:** *O(1)*

**Key Learning:** Fundamental application of the **Accumulator Pattern**. Initialize sums to `0`.

### Problem 4: Count Even Numbers in an Array

**Problem Statement:** Given an array of integers, return the count of numbers that are even.

**Approach:** Initialize `count = 0`. Loop through elements. If `arr[i] % 2 === 0`, increment `count`.

```
function countEvenNumbers(arr) {
  let count = 0;
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] % 2 === 0) {
      count++;
    }
  }
  return count;
}
```

**Time Complexity:** *O(n)* **Space Complexity:** *O(1)*

**Key Learning:** Demonstrates the **Counter Pattern** with a modulo condition check.

### Problem 5: Find the Sum of Odd Numbers in an Array

**Problem Statement:** Given an array of integers, return the sum of all odd numbers.

```
function sumOfOddNumbers(arr) {
  let oddSum = 0;
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] % 2 !== 0) {
      oddSum += arr[i];
    }
  }
  return oddSum;
}
```

**Time Complexity:** *O(n)* **Space Complexity:** *O(1)*

**Key Learning:** Combines condition filtering with accumulation in a single linear pass.

### Problem 6: Count Numbers Greater Than 20

**Problem Statement:** Return how many elements in an array are strictly greater than 20.

```
function countGreaterThan20(arr) {
  let count = 0;
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] > 20) {
      count++;
    }
  }
  return count;
}
```

**Time Complexity:** *O(n)* **Space Complexity:** *O(1)*

**Key Learning:** Pay careful attention to relational operators (strictly greater `>` vs. greater/equal `>=`).

## 6. Common Pitfalls & Avoidance Strategies

**1. Initializing Max/Min to 0**

Setting `let max = 0` fails when all array items are negative (e.g. `[-10, -20]` will return `0`).
**Fix:** Always initialize with `arr[0]`.

**2. Off-by-One Loop Bounds**

Writing `i <= arr.length` attempts to access `arr[arr.length]`, which is `undefined`.
**Fix:** Always loop with strict inequality `i < arr.length`.

**3. Modulo with Negative Odd Numbers in JS**

In JS, `-3 % 2` yields `-1`, not `1`. Checking `arr[i] % 2 === 1` fails for negative odds.
**Fix:** Use `arr[i] % 2 !== 0`.

## 7. Summary & Next Steps

**What I Learned Today**

- Understood Big O bounds ( `O(1)`, `O(log n)`, `O(n)`, `O(n²)` ).
- Mastered standard Array traversal and zero-based indexing mechanics.
- Implemented 4 basic design patterns: **Traversal**, **Comparison**, **Accumulator**, and **Counter**.
- Applied the systematic 6-step problem-solving process.

**Tomorrow's Topic (Day 2 Teaser)**

**Array Searching & Basic Operations:** Linear Search, Binary Search principles, insertion, deletion, and array reversing techniques!